DEVSYNC · DATABASE DESIGN

Logical & Physical Schema

Relational logical schema (3NF) · Cloud Firestore physical schema · PostgreSQL reference DDL

Project	**DevSync** — Secure AI Project Collaborator (Android · Flutter + Firebase)
Logical model	29 tables · 52 foreign-key relationships · 25 enumerations
Physical model	23 Firestore collection / subcollection paths · 28 composite indexes · 19 documented denormalizations
Sources	Core Features (core_features1.pdf) · DevSync Comprehensive Project Analysis (Requirements v1.2)
Companion file	DevSync_ERD.pdf (entity names and relationships match 1:1)
Date	29 October 2026

Contents

Part A — Logical Schema

A.1 Conventions and data types 4

A.2 Enumerations 4

A.3 Relational schema (compact notation) 6

A.4 Table definitions 7

A.5 Normalization to 3NF 18

B.1 Design principles 21

B.2 Collection tree 22

B.3 Collection and document specifications 23

B.4 Sample documents 33

B.5 Composite indexes 36

B.6 Denormalization register 38

B.7 Security rules 40

B.8 Firebase Storage 41

B.9 Logical → Physical mapping 42

C.1 DDL 43

D Assumptions & Open Points 50

Part A
Logical Schema

The logical schema is technology-independent: tables, columns, logical data types, keys and constraints, normalised to 3NF. No denormalization appears here - it is recorded only in the physical schema (Part B).

A.1 Conventions and data types

Logical type	PostgreSQL	Firestore	Notes
String(n)	`VARCHAR(n)`	string	IDs are strings (Firebase UID, Firestore doc ID)
Text	`TEXT`	string	free text, up to doc-size limit
Integer	`INTEGER`	number (int)	
Decimal(p,s)	`NUMERIC(p,s)`	number (double)	scores / averages
Boolean	`BOOLEAN`	boolean	
Timestamp	`TIMESTAMPTZ`	timestamp	stored in UTC
Date	`DATE`	timestamp	midnight UTC
Enum:name	`CREATE TYPE name AS ENUM`	string	validated by Security Rules / app

Column "Key"	Meaning
PK	Primary key (composite when several columns are marked PK).
FK	Foreign key — see "References" and the ON DELETE rule.
UK	Unique (single column). Composite unique constraints are listed under each table.
Null = Y	Column is optional (NULL allowed). Otherwise NOT NULL.

ON DELETE rule: a mandatory FK uses CASCADE (deleting the parent deletes the children, which implements Delete account), an optional FK uses SET NULL, and the exceptions use RESTRICT (JOIN_REQUEST.role_id, AI_GENERATION.prompt_version_id, PROMPT_EVAL_RUN.prompt_version_id) so roles and prompt versions that are in use cannot be deleted.

A.2 Enumerations

Enum	Values
`track`	`Mobile | Web | Backend | Data | AI | UI_UX | QA | DevOps | Other`
`level`	`Beginner | Junior | Intermediate | Advanced`
`auth_provider`	`password | google`
`platform`	`android`
`skill_kind`	`PROGRAMMING_LANGUAGE | TECHNICAL | SOFT | INTEREST`
`skill_source`	`manual | github`
`evidence_type`	`Repo | Commits | Pull_Request | Language_Usage`
`detect_status`	`pending | accepted | rejected`
`project_purpose`	`Learning | Real_World`
`project_type`	`Web_App | Mobile_App | API_Backend | Library_Tool | Data_AI | Game | Other`
`difficulty`	`Beginner | Intermediate | Advanced`
`project_status`	`Draft | Open | Full | Completed | Archived`
`creation_mode`	`Manual | AI_Assisted`
`member_type`	`Owner | Member`
`member_status`	`Active | Left | Removed`
`request_status`	`Pending | Accepted | Rejected`
`invitation_status`	`Pending | Accepted | Declined | Cancelled`
`engine`	`LLM | RULE_BASED`
`feature_type`	`SKILL_INFERENCE | MATCH_EXPLAIN | SKILL_GAP | PROJECT_SETUP | ROLE_GENERATOR | COLLAB_PITCH`
`gen_source`	`Live | Cached | Fallback`

match_type	Matched | Complementary | Cross_Skill | Missing
task_status	To_Do | In_Progress | In_Review | Done
task_priority	Low | Medium | High
conversation_type	Direct | Team
notification_type	Join_Request_Received | Request_Accepted | Request_Rejected | Invitation_Received | Task_Assigned | Task_Updated | New_Message | Missing_Role

Values that come from the documents: level, task_status, request_status, project_purpose, detect_status. All other values are proposals (see Assumptions).

A.3 Relational schema (compact notation)

Notation: underline = PK, *italic* = FK, ? = nullable.

users(uid, email, email_verified, auth_provider, display_name, public_id, photo_url?, bio?, main_track?, level?, background?, cv_url?, portfolio_url?, linkedin_url?, behance_url?, created_at, updated_at)

user_devices(device_id, *user_id*, fcm_token, platform, last_seen_at)

skills(skill_id, name, kind, is_active)

user_skills(*user_id*, *skill_id*, proficiency?, source, added_at)

github_data(*user_id*, github_username, repos_count, commits_count, pull_requests_count, last_activity_at?, last_sync_at, sync_status)

github_repos(repo_id, *user_id*, repo_name, primary_language?, stars, commit_count, last_pushed_at?, repo_url)

detected_skills(detected_id, *user_id*, *skill_id*, confidence, last_active_at?, status, detected_at, reviewed_at?)

skill_evidence(evidence_id, *detected_id*, *repo_id*?, evidence_type, summary, commit_count?, last_activity_at?)

projects(project_id, *owner_id*, title, description, purpose, project_type, track, difficulty, status, team_size_target, expected_hours_per_week?, min_level?, mentoring_available?, expected_end_date?, creation_mode, *setup_generation_id*?, created_at, published_at?, updated_at)

project_skills(*project_id*, *skill_id*, is_required)

project_roles(role_id, *project_id*, role_name, description?, slots_needed, is_ai_generated, created_at)

role_skills(*role_id*, *skill_id*)

project_members(*project_id*, *user_id*, *role_id*?, member_type, status, joined_at, left_at?)

saved_projects(*user_id*, *project_id*, saved_at)

interests(*user_id*, *project_id*, created_at)

join_requests(request_id, *project_id*, *applicant_id*, *role_id*, pitch_message, *pitch_generation_id*?, status, created_at, decided_at?)

invitations(invitation_id, *project_id*, *role_id*?, *invitee_id*, *inviter_id*, message?, status, created_at, responded_at?)

prompt_versions(prompt_version_id, feature_type, version_label, engine, template?, is_active, created_at, notes?)

ai_generations(generation_id, *user_id*, *prompt_version_id*, source, latency_ms?, created_at)

match_recommendations(match_id, *user_id*, *project_id*, *generation_id*, match_score, explanation_text, created_at)

match_skills(*match_id*, *skill_id*, match_type, learning_tip?)

ai_feedback(feedback_id, *generation_id*, *user_id*, is_helpful, clarity_rating?, reason?, created_at)

prompt_eval_runs(run_id, *prompt_version_id*, test_set_label, sample_count, avg_relevance, avg_clarity, suitable_rate_pct, avg_latency_ms?, run_at, notes?)

tasks(task_id, *project_id*, title, description?, status, priority, deadline?, *assignee_id*?, *creator_id*, created_at, updated_at)

conversations(conversation_id, conversation_type, *project_id*?, created_at)

conversation_participants(*conversation_id*, *user_id*, joined_at, last_read_at?)

messages(message_id, *conversation_id*, *sender_id*, body, sent_at, edited_at?)

notifications(notification_id, *user_id*, type, title, body, *project_id*?, deep_link?, is_read, created_at)

ai_usage_counters(*user_id*, usage_date, request_count)

A.4 Table definitions

Each table with its columns, types, keys, constraints and FK references.

A.4.1 users (entity USER) Identity & Profile

A developer / project owner account: account data and the public profile. A Guest is not stored here (no account, nothing persisted).

Column	Type	Key	Null	References	Notes
uid	String(128)	PK			Firebase Auth UID
email	String(255)	UK			From Firebase Auth
email_verified	Boolean				Email verification state
auth_provider	Enum(auth_provider)				Sign-up method (Email / Google)
display_name	String(80)				Mandatory name
public_id	String(20)	UK			Auto-generated unique user ID shown on the profile
photo_url	String(500)		Y		Profile photo
bio	Text		Y		
main_track	Enum(track)		Y		Mandatory once the profile is completed
level	Enum(level)		Y		Mandatory once the profile is completed
background	Text		Y		Other technical background (free text)
cv_url	String(500)		Y		CV path in Firebase Storage
portfolio_url	String(500)		Y		
linkedin_url	String(500)		Y		Link only (not a sign-in method)
behance_url	String(500)		Y		Shown only when connected
created_at	Timestamp				
updated_at	Timestamp				

PRIMARY KEY (uid) | CHECK: main_track, level and at least one programming language are required when the profile is completed (validated in the app - US-02)

A.4.2 user_devices (entity USER_DEVICE) Identity & Profile

User devices and their FCM token, used to deliver push notifications (one or more devices per user).

Column	Type	Key	Null	References	Notes
device_id	String(64)	PK			
user_id	String(128)	FK		users(uid) ON DELETE CASCADE	
fcm_token	String(255)	UK			
platform	Enum(platform)				Android only (project scope)
last_seen_at	Timestamp				

PRIMARY KEY (device_id)

A.4.3 skills (entity SKILL) Identity & Profile

Skill catalogue (programming languages, technical skills, soft skills, interests) shared by profiles and projects.

Column	Type	Key	Null	References	Notes
skill_id	String(64)	PK			
name	String(80)				Skill name
kind	Enum(skill_kind)				Skill kind
is_active	Boolean				

PRIMARY KEY (skill_id) | UNIQUE (name, kind)

A.4.4 user_skills (entity USER_SKILL) Identity & Profile

Associative table (M:N) between users and their confirmed skills. Each programming language carries a level.

Column	Type	Key	Null	References	Notes
user_id	String(128)	PK · FK		users(uid) ON DELETE CASCADE	
skill_id	String(64)	PK · FK		skills(skill_id) ON DELETE CASCADE	
proficiency	Enum(level)		Y		Language level (programming languages only)
source	Enum(skill_source)				manual / github (accepted from AI-Detected skills)
added_at	Timestamp				

PRIMARY KEY (user_id, skill_id)

A.4.5 github_data (entity GITHUB_DATA) GitHub Integration

GitHub connection summary for a user (1:1). The OAuth token is NOT stored here (NFR-07).

Column	Type	Key	Null	References	Notes
user_id	String(128)	PK · FK		users(uid) ON DELETE CASCADE	
github_username	String(80)	UK			
repos_count	Integer				
commits_count	Integer				
pull_requests_count	Integer				
last_activity_at	Timestamp		Y		Latest activity
last_sync_at	Timestamp				Re-sync
sync_status	String(20)				ok / rate_limited / failed

PRIMARY KEY (user_id)

A.4.6 github_repos (entity GITHUB_REPO) GitHub Integration

Repository summaries pulled from GitHub (top languages are computed from them).

Column	Type	Key	Null	References	Notes
repo_id	String(64)	PK			
user_id	String(128)	FK		github_data(user_id) ON DELETE CASCADE	
repo_name	String(120)				
primary_language	String(40)		Y		
stars	Integer				
commit_count	Integer				
last_pushed_at	Timestamp		Y		
repo_url	String(500)				

PRIMARY KEY (repo_id) | UNIQUE (user_id, repo_name)

A.4.7 detected_skills (entity DETECTED_SKILL) GitHub Integration

A skill inferred by the AI from GitHub. It is NOT a verified fact until the user accepts it (status).

Column	Type	Key	Null	References	Notes
detected_id	String(64)	PK			
user_id	String(128)	FK		users(uid) ON DELETE CASCADE	
skill_id	String(64)	FK		skills(skill_id) ON DELETE CASCADE	
confidence	Decimal(3,2)				0.00 - 1.00
last_active_at	Timestamp		Y		
status	Enum(detect_status)				pending / accepted / rejected
detected_at	Timestamp				
reviewed_at	Timestamp		Y		

PRIMARY KEY (detected_id) | UNIQUE (user_id, skill_id) | CHECK: confidence BETWEEN 0 AND 1

A.4.8 skill_evidence (entity SKILL_EVIDENCE) GitHub Integration

Evidence backing an inferred skill (repo / commits / PR). Resolves the multi-valued attribute.

Column	Type	Key	Null	References	Notes
evidence_id	String(64)	PK			
detected_id	String(64)	FK		detected_skills(detected_id) ON DELETE CASCADE	
repo_id	String(64)	FK	Y	github_repos(repo_id) ON DELETE SET NULL	
evidence_type	Enum(evidence_type)				
summary	String(255)				
commit_count	Integer		Y		
last_activity_at	Timestamp		Y		

PRIMARY KEY (evidence_id)

A.4.9 projects (entity PROJECT) Projects & Teams

A project (created manually or AI-assisted). Project Purpose is mandatory and drives related fields.

Column	Type	Key	Null	References	Notes
project_id	String(64)	PK			
owner_id	String(128)	FK		users(uid) ON DELETE CASCADE	Project owner
title	String(120)				
description	Text				
purpose	Enum(project_purpose)				Learning / Real_World (mandatory)
project_type	Enum(project_type)				
track	Enum(track)				
difficulty	Enum(difficulty)				
status	Enum(project_status)				Draft / Open / Full / Completed / Archived
team_size_target	Integer				Target team size
expected_hours_per_week	Integer		Y		Expected commitment (Real-world)
min_level	Enum(level)		Y		Minimum level (Real-world)
mentoring_available	Boolean		Y		Mentoring available (Learning)
expected_end_date	Date		Y		Expected end date
creation_mode	Enum(creation_mode)				Manual / AI_Assisted
setup_generation_id	String(64)	FK · UK	Y	ai_generations(generation_id) ON DELETE SET NULL	AI generation of the setup (when used)
created_at	Timestamp				
published_at	Timestamp		Y		Published only with the owner's approval
updated_at	Timestamp				

PRIMARY KEY (project_id) | CHECK: purpose = 'Real_World' => expected_hours_per_week and min_level NOT NULL | CHECK: purpose = 'Learning' => no strict deadline (expected_end_date optional) | CHECK: team_size_target >= 1

A.4.10 project_skills (entity PROJECT_SKILL) Projects & Teams

Associative table (M:N): the project's tech stack and required skills.

Column	Type	Key	Null	References	Notes
project_id	String(64)	PK · FK		projects(project_id) ON DELETE CASCADE	
skill_id	String(64)	PK · FK		skills(skill_id) ON DELETE CASCADE	
is_required	Boolean				Required vs nice-to-have

PRIMARY KEY (project_id, skill_id)

A.4.11 project_roles (entity PROJECT_ROLE) Projects & Teams

Roles a project needs (Frontend, Backend, QA ...). AI-generated and editable.

Column	Type	Key	Null	References	Notes
role_id	String(64)	PK			
project_id	String(64)	FK		projects(project_id) ON DELETE CASCADE	
role_name	String(60)				
description	Text		Y		
slots_needed	Integer				Number of open places (default 1)
is_ai_generated	Boolean				
created_at	Timestamp				

PRIMARY KEY (role_id) | UNIQUE (project_id, role_name) | CHECK: slots_needed >= 1

A.4.12 role_skills (entity ROLE_SKILL) Projects & Teams

Associative table: skills required by each role (supports Missing Role Alert and Skill Gap). Design addition.

Column	Type	Key	Null	References	Notes
role_id	String(64)	PK · FK		project_roles(role_id) ON DELETE CASCADE	
skill_id	String(64)	PK · FK		skills(skill_id) ON DELETE CASCADE	

PRIMARY KEY (role_id, skill_id)

A.4.13 project_members (entity PROJECT_MEMBER) Projects & Teams

Team membership (associative between user and project). The owner is recorded as an Owner member.

Column	Type	Key	Null	References	Notes
project_id	String(64)	PK · FK		projects(project_id) ON DELETE CASCADE	
user_id	String(128)	PK · FK		users(uid) ON DELETE CASCADE	
role_id	String(64)	FK	Y	project_roles(role_id) ON DELETE SET NULL	Role the member holds
member_type	Enum(member_type)				Owner / Member
status	Enum(member_status)				Active / Left / Removed
joined_at	Timestamp				
left_at	Timestamp		Y		

PRIMARY KEY (project_id, user_id)

A.4.14 saved_projects (entity SAVED_PROJECT) Projects & Teams

Saved projects. Save and Favorites are merged into a single Saved list.

Column	Type	Key	Null	References	Notes
user_id	String(128)	PK · FK		users(uid) ON DELETE CASCADE	
project_id	String(64)	PK · FK		projects(project_id) ON DELETE CASCADE	
saved_at	Timestamp				

PRIMARY KEY (user_id, project_id)

A.4.15 interests (entity INTEREST) Projects & Teams

Express Interest: an action separate from Save and Request to Join. At most once per user and project.

Column	Type	Key	Null	References	Notes
user_id	String(128)	PK · FK		users(uid) ON DELETE CASCADE	
project_id	String(64)	PK · FK		projects(project_id) ON DELETE CASCADE	
created_at	Timestamp				

PRIMARY KEY (user_id, project_id)

A.4.16 join_requests (entity JOIN_REQUEST) Collaboration & Workspace

A request to join a project. Sent once only (UNIQUE) with status Pending / Accepted / Rejected.

Column	Type	Key	Null	References	Notes
request_id	String(64)	PK			
project_id	String(64)	FK		projects(project_id) ON DELETE CASCADE	
applicant_id	String(128)	FK		users(uid) ON DELETE CASCADE	Applicant
role_id	String(64)	FK		project_roles(role_id) ON DELETE RESTRICT	Requested role
pitch_message	Text				Join message (AI-generated, editable)
pitch_generation_id	String(64)	FK · UK	Y	ai_generations(generation_id) ON DELETE SET NULL	
status	Enum(request_status)				
created_at	Timestamp				
decided_at	Timestamp		Y		

PRIMARY KEY (request_id) | UNIQUE (project_id, applicant_id)

A.4.17 invitations (entity INVITATION) Collaboration & Workspace

An invitation sent to a developer (Invite Developer). The owner can cancel it.

Column	Type	Key	Null	References	Notes
invitation_id	String(64)	PK			
project_id	String(64)	FK		projects(project_id) ON DELETE CASCADE	
role_id	String(64)	FK	Y	project_roles(role_id) ON DELETE SET NULL	
invitee_id	String(128)	FK		users(uid) ON DELETE CASCADE	Invited developer
inviter_id	String(128)	FK		users(uid) ON DELETE CASCADE	Owner who sent the invitation
message	Text		Y		
status	Enum(invitation_status)				
created_at	Timestamp				
responded_at	Timestamp		Y		

PRIMARY KEY (invitation_id) | CHECK: UNIQUE (project_id, invitee_id) WHERE status = 'Pending' (partial index)

A.4.18 prompt_versions (entity PROMPT_VERSION) AI, Matching & Evaluation

Prompt versions (v1, v2 ...) per generation type. The rule-based fallback is also recorded as a version.

Column	Type	Key	Null	References	Notes
prompt_version_id	String(64)	PK			
feature_type	Enum(feature_type)				
version_label	String(20)				v1, v2 ...
engine	Enum(engine)				LLM / RULE_BASED
template	Text		Y		Prompt text
is_active	Boolean				Currently active version (Remote Config)
created_at	Timestamp				
notes	Text		Y		

PRIMARY KEY (prompt_version_id) | UNIQUE (feature_type, version_label)

A.4.19 ai_generations (entity AI_GENERATION) AI, Matching & Evaluation

Log of every AI output (recommendation, project setup, pitch). Links an output to its prompt version.

Column	Type	Key	Null	References	Notes
generation_id	String(64)	PK			
user_id	String(128)	FK		users(uid) ON DELETE CASCADE	User who requested the generation
prompt_version_id	String(64)	FK		prompt_versions(prompt_version_id) ON DELETE RESTRICT	
source	Enum(gen_source)				Live / Cached / Fallback (NFR-11)
latency_ms	Integer		Y		Response time (NFR-01)
created_at	Timestamp				

PRIMARY KEY (generation_id)

A.4.20 match_recommendations (entity MATCH_RECOMMENDATION) AI, Matching & Evaluation

A project recommendation for a user plus its explanation. All rows together form the Match History.

Column	Type	Key	Null	References	Notes
match_id	String(64)	PK			
user_id	String(128)	FK		users(uid) ON DELETE CASCADE	
project_id	String(64)	FK		projects(project_id) ON DELETE CASCADE	
generation_id	String(64)	FK · UK		ai_generations(generation_id) ON DELETE CASCADE	
match_score	Integer				0 - 100
explanation_text	Text				Natural-language explanation (never a bare score)
created_at	Timestamp				

PRIMARY KEY (match_id) | CHECK: match_score BETWEEN 0 AND 100

A.4.21 match_skills (entity MATCH_SKILL) AI, Matching & Evaluation

Per-recommendation detail: matched / complementary / close / missing skills, with a short learning tip for missing ones.

Column	Type	Key	Null	References	Notes
match_id	String(64)	PK · FK		match_recommendations(match_id) ON DELETE CASCADE	
skill_id	String(64)	PK · FK		skills(skill_id) ON DELETE CASCADE	
match_type	Enum(match_type)				Matched / Complementary / Cross_Skill / Missing
learning_tip	Text		Y		Only for Missing

PRIMARY KEY (match_id, skill_id) | CHECK: learning_tip is allowed only when match_type = 'Missing'

A.4.22 ai_feedback (entity AI_FEEDBACK) AI, Matching & Evaluation

User rating of an AI output: thumbs up/down + clarity 1-5 + optional reason.

Column	Type	Key	Null	References	Notes
feedback_id	String(64)	PK			
generation_id	String(64)	FK		ai_generations(generation_id) ON DELETE CASCADE	
user_id	String(128)	FK		users(uid) ON DELETE CASCADE	
is_helpful	Boolean				Thumbs up / down
clarity_rating	Integer		Y		1 - 5
reason	Text		Y		Reason for rejection
created_at	Timestamp				

PRIMARY KEY (feedback_id) | UNIQUE (generation_id, user_id) | CHECK: clarity_rating BETWEEN 1 AND 5

A.4.23 prompt_eval_runs (entity PROMPT_EVAL_RUN) AI, Matching & Evaluation

Result of running a prompt version against a fixed test set (the required delivery evidence).

Column	Type	Key	Null	References	Notes
run_id	String(64)	PK			
prompt_version_id	String(64)	FK		prompt_versions(prompt_version_id) ON DELETE RESTRICT	
test_set_label	String(60)				Test set name
sample_count	Integer				
avg_relevance	Decimal(3,2)				0 - 5
avg_clarity	Decimal(3,2)				Explanation clarity (target >= 4)
suitable_rate_pct	Decimal(5,2)				% of recommendations rated suitable (target >= 60)
avg_latency_ms	Integer		Y		
run_at	Timestamp				
notes	Text		Y		

PRIMARY KEY (run_id)

A.4.24 tasks (entity TASK) Collaboration & Workspace

A task on the board: To Do / In Progress / In Review / Done.

Column	Type	Key	Null	References	Notes
task_id	String(64)	PK			
project_id	String(64)	FK		projects(project_id) ON DELETE CASCADE	
title	String(150)				
description	Text		Y		
status	Enum(task_status)				
priority	Enum(task_priority)				
deadline	Date		Y		
assignee_id	String(128)	FK	Y	users(uid) ON DELETE SET NULL	Assignee (must be a team member)
creator_id	String(128)	FK		users(uid) ON DELETE CASCADE	
created_at	Timestamp				
updated_at	Timestamp				

PRIMARY KEY (task_id)

A.4.25 conversations (entity CONVERSATION) Messaging

A direct (1-to-1) conversation or a team conversation tied to a project.

Column	Type	Key	Null	References	Notes
conversation_id	String(64)	PK			
conversation_type	Enum(conversation_type)				
project_id	String(64)	FK · UK	Y	projects(project_id) ON DELETE SET NULL	Team conversations only
created_at	Timestamp				

PRIMARY KEY (conversation_id) | CHECK: (conversation_type='Team') = (project_id IS NOT NULL)

A.4.26 conversation_participants (entity CONVERSATION_PARTICIPANT) Messaging

Associative table (M:N) between conversations and users, plus last-read time.

Column	Type	Key	Null	References	Notes
conversation_id	String(64)	PK · FK		conversations(conversation_id) ON DELETE CASCADE	
user_id	String(128)	PK · FK		users(uid) ON DELETE CASCADE	
joined_at	Timestamp				
last_read_at	Timestamp		Y		

PRIMARY KEY (conversation_id, user_id)

A.4.27 messages (entity MESSAGE) Messaging

A message inside a conversation (real-time).

Column	Type	Key	Null	References	Notes
message_id	String(64)	PK			
conversation_id	String(64)	FK		conversations(conversation_id) ON DELETE CASCADE	
sender_id	String(128)	FK		users(uid) ON DELETE CASCADE	
body	Text				
sent_at	Timestamp				
edited_at	Timestamp		Y		

PRIMARY KEY (message_id)

A.4.28 notifications (entity NOTIFICATION) Collaboration & Workspace

An in-app notification (Today / Earlier list) with a reference for FCM and the deep link.

Column	Type	Key	Null	References	Notes
notification_id	String(64)	PK			
user_id	String(128)	FK		users(uid) ON DELETE CASCADE	Recipient
type	Enum(notification_type)				
title	String(120)				
body	String(300)				
project_id	String(64)	FK	Y	projects(project_id) ON DELETE SET NULL	
deep_link	String(255)		Y		
is_read	Boolean				
created_at	Timestamp				

PRIMARY KEY (notification_id)

A.4.29 ai_usage_counters (entity AI_USAGE_COUNTER) AI, Matching & Evaluation

Daily counter of AI requests per user (cost control).

Column	Type	Key	Null	References	Notes
user_id	String(128)	PK · FK		users(uid) ON DELETE CASCADE	
usage_date	Date	PK			
request_count	Integer				

PRIMARY KEY (user_id, usage_date)

A.5 Normalization to 3NF

1NF: every column holds a single atomic value and there are no arrays. Multi-valued attributes were moved into tables: user skills -> `user_skills`, skill evidence -> `skill_evidence`, matched / missing skills -> `match_skills`, conversation participants -> `conversation_participants`.

2NF: in tables with a composite key (user_skills, project_skills, role_skills, project_members, saved_projects, interests, match_skills, conversation_participants, ai_usage_counters) every non-key column depends on the whole key, not on part of it.

3NF: there are no transitive dependencies. The table below lists the main decisions taken to reach 3NF.

Table(s)	Normalization decision	Result
`users`	Derived values (profile_completion_pct, github_connected, has_cv) were removed: they are computable from other columns / tables.	**3NF**
`projects`	Owner name/photo are not copied (owner_id → users). Purpose-specific columns depend on project_id, not on purpose, and are guarded with CHECK constraints.	**3NF**
`project_skills` / `role_skills` / `user_skills`	Skill name and kind live only in skills; the link tables hold only link attributes (is_required, proficiency, source).	**3NF**
`ai_generations`	feature_type is NOT stored here: prompt_version_id → feature_type is a transitive dependency, so it stays in prompt_versions (rule-based fallbacks are rows in prompt_versions with engine = RULE_BASED).	**3NF**
`match_recommendations`	source / latency / prompt version are in ai_generations (1:1). Matched / missing skills are in match_skills. match_score and explanation depend only on match_id.	**3NF**
`ai_feedback`	One row per (generation_id, user_id). It references ai_generations instead of the target (match / pitch / setup / chat) so there is no polymorphic FK.	**3NF**
`project_members`	Owner info is not duplicated; member_type, status, joined_at depend on the whole key (project_id, user_id).	**3NF**
`join_requests`	Decision maker = projects.owner_id (not stored). UNIQUE (project_id, applicant_id) enforces one request only.	**3NF**
`conversations`	last_message / participants list / unread counters are derived → excluded (physical layer only).	**3NF**
`notifications`	project_id + deep_link only; no copies of project title or sender name.	**3NF**
`detected_skills` / `skill_evidence`	One detection per (user, skill); evidence rows are separate because one skill has many pieces of evidence.	**3NF**

Functional dependencies per table

For each table the key determines every other column; tables with extra candidate keys list them.

Table	Primary-key dependency	Other candidate keys
users	{uid} → {email, email_verified, auth_provider, display_name, public_id, photo_url, bio, main_track, level, background, cv_url, portfolio_url, linkedin_url, behance_url, created_at, updated_at}	email → all; public_id → all
user_devices	{device_id} → {user_id, fcm_token, platform, last_seen_at}	fcm_token → all
skills	{skill_id} → {name, kind, is_active}	{name,kind} → all
user_skills	{user_id, skill_id} → {proficiency, source, added_at}	—
github_data	{user_id} → {github_username, repos_count, commits_count, pull_requests_count, last_activity_at, last_sync_at, sync_status}	github_username → all
github_repos	{repo_id} → {user_id, repo_name, primary_language, stars, commit_count, last_pushed_at, repo_url}	{user_id,repo_name} → all
detected_skills	{detected_id} → {user_id, skill_id, confidence, last_active_at, status, detected_at, reviewed_at}	{user_id,skill_id} → all
skill_evidence	{evidence_id} → {detected_id, repo_id, evidence_type, summary, commit_count, last_activity_at}	—

projects	{project_id} → {owner_id, title, description, purpose, project_type, track, difficulty, status, team_size_target, expected_hours_per_week, min_level, mentoring_available, expected_end_date, creation_mode, setup_generation_id, created_at, published_at, updated_at}	setup_generation_id → all
project_skills	{project_id, skill_id} → {is_required}	—
project_roles	{role_id} → {project_id, role_name, description, slots_needed, is_ai_generated, created_at}	{project_id,role_name} → all
role_skills	{role_id, skill_id} → {∅}	—
project_members	{project_id, user_id} → {role_id, member_type, status, joined_at, left_at}	—
saved_projects	{user_id, project_id} → {saved_at}	—
interests	{user_id, project_id} → {created_at}	—
join_requests	{request_id} → {project_id, applicant_id, role_id, pitch_message, pitch_generation_id, status, created_at, decided_at}	pitch_generation_id → all; {project_id,applicant_id} → all
invitations	{invitation_id} → {project_id, role_id, invitee_id, inviter_id, message, status, created_at, responded_at}	—
prompt_versions	{prompt_version_id} → {feature_type, version_label, engine, template, is_active, created_at, notes}	{feature_type,version_label} → all
ai_generations	{generation_id} → {user_id, prompt_version_id, source, latency_ms, created_at}	—
match_recommendations	{match_id} → {user_id, project_id, generation_id, match_score, explanation_text, created_at}	generation_id → all
match_skills	{match_id, skill_id} → {match_type, learning_tip}	—
ai_feedback	{feedback_id} → {generation_id, user_id, is_helpful, clarity_rating, reason, created_at}	{generation_id,user_id} → all
prompt_eval_runs	{run_id} → {prompt_version_id, test_set_label, sample_count, avg_relevance, avg_clarity, suitable_rate_pct, avg_latency_ms, run_at, notes}	—
tasks	{task_id} → {project_id, title, description, status, priority, deadline, assignee_id, creator_id, created_at, updated_at}	—
conversations	{conversation_id} → {conversation_type, project_id, created_at}	project_id → all
conversation_participants	{conversation_id, user_id} → {joined_at, last_read_at}	—
messages	{message_id} → {conversation_id, sender_id, body, sent_at, edited_at}	—
notifications	{notification_id} → {user_id, type, title, body, project_id, deep_link, is_read, created_at}	—
ai_usage_counters	{user_id, usage_date} → {request_count}	—

Part B
Physical Schema

Cloud Firestore

The project database is Firebase Firestore (NoSQL), so the physical schema describes collections, subcollections, document IDs, Firestore data types, composite indexes, denormalization and Security Rules. A PostgreSQL version is given in Part C for reference.

B.1 Design principles

- Query-first: the schema is designed around the screens and queries (Discover, Incoming Requests, My Teams, Inbox, Task Board), not around tables.
- References are stored as **string IDs** instead of the DocumentReference type: easier for Security Rules, queries and migration. The logical type is stated in the Notes column.
- **Subcollections** hold data that is always read with its parent (roles, members, tasks, messages, savedProjects, notifications). **Top-level collections** hold data read from several directions (joinRequests, invitations, conversations).
- **Deterministic IDs** enforce the UNIQUE constraints of the logical schema: joinRequests/{projectId}_{applicantId}, members/{uid}, savedProjects/{projectId}, aiFeedback/{generationId}_{userId}, detectedSkills/{skillId}.
- Denormalization is intentional and fully recorded in B.6 with its reason and how it is refreshed (Cloud Functions triggers). Sensitive writes (matches, aiGenerations, counters, members) are done by Cloud Functions only (Admin SDK).
- Firestore limits that were taken into account: a document is at most 1 MiB (hence repos <= 30); one query may use only one array-contains / array-contains-any; array-contains-any accepts at most 30 values. The Tech stack and Roles needed filters therefore cannot both run server-side in one query - the second is applied client-side or in a Cloud Function (assumption).
- A Guest is read-only: skills, projects (!= Draft), roles and users (public profile). No email or CV URL is stored in a public document.
- All timestamps are stored in UTC and displayed in the device time zone.

B.2 Collection tree

```
skills/{skillId}
users/{uid}
    └ private/account
    └ devices/{deviceId}
    └ githubData/summary
    └ detectedSkills/{skillId}
    └ savedProjects/{projectId}
    └ matches/{matchId}
    └ notifications/{notificationId}
    └ aiUsage/{yyyyMMdd}
projects/{projectId}
    └ roles/{roleId}
    └ members/{uid}
    └ tasks/{taskId}
    └ interests/{uid}
joinRequests/{projectId}_{applicantId}
invitations/{projectId}_{inviteeId}
conversations/{conversationId}
    └ messages/{messageId}
promptVersions/{promptVersionId}
promptEvalRuns/{runId}
aiGenerations/{generationId}
aiFeedback/{generationId}_{userId}
```

The Guest has no collection of its own, and there is no collection for tags or posts (those features are out of scope). Subcollections are listed under their parent document.

B.3 Collection and document specifications

Orange rows are denormalized (D) fields or fields transformed from another entity; the reasons are in B.6.
Req: required / optional.

B.3.1 `skills/{skillId}`

Logical source: **SKILL**

Catalogue readable by everyone (guests included); written only by admins / Cloud Functions.

Field	Firestore type	Req	Source / notes
`(document ID)`	string	req	Auto-ID or slug (e.g. flutter) - stable
`name`	string	req	Skill name
`kind`	string (enum skill_kind)	req	Skill kind
`isActive`	boolean	req	

B.3.2 `users/{uid}`

Logical source: **USER** · Subcollections: private, devices, githubData, detectedSkills, savedProjects, matches, notifications, aiUsage

This document is publicly readable (guests browse Public Profiles), so it holds no email and no CV URL.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Firebase Auth UID
authProvider	string (enum auth_provider)	req	Sign-up method (Email / Google)
displayName	string	req	Mandatory name
publicId	string	req	Auto-generated unique user ID shown on the profile
photoUrl	string	opt	Profile photo
bio	string	opt	
mainTrack	string (enum track)	opt	Mandatory once the profile is completed
level	string (enum level)	opt	Mandatory once the profile is completed
background	string	opt	Other technical background (free text)
portfolioUrl	string	opt	
linkedinUrl	string	opt	Link only (not a sign-in method)
behanceUrl	string	opt	Shown only when connected
createdAt	timestamp	req	
updatedAt	timestamp	req	
languages D	array<map>	opt	{skillId, name, level} - USER_SKILL (kind = PROGRAMMING_LANGUAGE)
skills D	array<map>	opt	{skillId, name} - USER_SKILL (kind = TECHNICAL)
softSkills D	array<map>	opt	{skillId, name} - USER_SKILL (kind = SOFT)
interests D	array<map>	opt	{skillId, name} - USER_SKILL (kind = INTEREST)
skillIds D	array<string>	opt	Flat list of all skill IDs (array-contains in Discover Developers)
githubConnected D	boolean	req	= users/{uid}/githubData/summary exists
hasCv D	boolean	req	= cvUrl exists in private/account (the URL itself is not public)
profileCompletion D	number	req	0-100, computed from the profile fields (Profile Completion)

B.3.3 `users/{uid}/private/account`

Logical source: **USER (private part)**

Vertical partition for private data (owner only).

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Fixed: account
email	string	req	From Firebase Auth (USER.email)
cvUrl	string	opt	File path in Firebase Storage (USER.cv_url)

B.3.4 `users/{uid}/devices/{deviceId}`

Logical source: **USER_DEVICE**

Owner only.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Installation ID
fcmToken	string	req	
platform	string (enum platform)	req	Android only (project scope)
lastSeenAt	timestamp	req	

B.3.5 `users/{uid}/githubData/summary`

Logical source: **GITHUB_DATA**

The token is not stored here (NFR-07): secure storage or server-side (Secret Manager).

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Fixed: summary
githubUsername	string	req	
reposCount	number (int)	req	
commitsCount	number (int)	req	
pullRequestsCount	number (int)	req	
lastActivityAt	timestamp	opt	Latest activity
lastSyncAt	timestamp	req	Re-sync
syncStatus	string	req	ok / rate_limited / failed
languages **D**	map<string, number>	opt	Top languages % - computed from GITHUB_REPO.primary_language
repos **D**	array<map>	opt	GITHUB_REPO: {repoId, name, primaryLanguage, stars, commitCount, lastPushedAt, url} (latest <= 30 repos)

B.3.6 `users/{uid}/detectedSkills/{skillId}`

Logical source: **DETECTED_SKILL**

status = pending until the user accepts or rejects. Accepting copies the skill into users/{uid} skills / languages.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	skillId (guarantees UNIQUE user + skill)
confidence	number (double)	req	0.00 - 1.00
lastActiveAt	timestamp	opt	
status	string (enum detect_status)	req	pending / accepted / rejected
detectedAt	timestamp	req	
reviewedAt	timestamp	opt	
skillName **D**	string	req	Skill name (SKILL.name)
evidence **D**	array<map>	req	SKILL_EVIDENCE: {type, repoName, summary, commitCount, lastActivityAt}

B.3.7 `users/{uid}/savedProjects/{projectId}`

Logical source: **SAVED_PROJECT**

Field	Firestore type	Req	Source / notes
(document ID)	string	req	projectId (guarantees UNIQUE)
savedAt	timestamp	req	
title **D**	string	req	Snapshot of PROJECT.title
purpose **D**	string	req	Snapshot
difficulty **D**	string	req	Snapshot
status **D**	string	req	Snapshot (refreshed by a Function)

B.3.8 `users/{uid}/matches/{matchId}`

Logical source: **MATCH_RECOMMENDATION**

Match History = all documents here ordered by createdAt. Written by Cloud Functions only.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Auto-ID
projectId	string	req	FK → projects
matchScore	number (int)	req	0 - 100
explanationText	string	req	Natural-language explanation (never a bare score)
createdAt	timestamp	req	
generationId **FK**	string	req	FK -> aiGenerations (rating reference)
projectTitle **D**	string	req	Snapshot of PROJECT.title
projectPurpose **D**	string	req	Snapshot
matchedSkills **D**	array<map>	req	MATCH_SKILL (Matched): {skillId, name}
complementarySkills **D**	array<map>	opt	MATCH_SKILL (Complementary)
crossSkills **D**	array<map>	opt	MATCH_SKILL (Cross_Skill): skills close to the required ones
missingSkills **D**	array<map>	opt	MATCH_SKILL (Missing): {skillId, name, learningTip}
source **D**	string	req	AI_GENERATION.source (Live / Cached / Fallback)
promptVersionId **D**	string	req	AI_GENERATION.prompt_version_id

B.3.9 `users/{uid}/notifications/{notificationId}`

Logical source: **NOTIFICATION**

Bell icon + Today / Earlier list. Functions create; the user reads and sets isRead.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Auto-ID
type	string (enum notification_type)	req	
title	string	req	
body	string	req	
projectId	string	opt	FK → projects
deepLink	string	opt	
isRead	boolean	req	
createdAt	timestamp	req	

B.3.10 `users/{uid}/aiUsage/{yyyyMMdd}`

Logical source: **AI_USAGE_COUNTER**

Incremented in a transaction inside a Cloud Function before any LLM call.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Date as yyyyMMdd
requestCount	number (int)	req	

B.3.11 `projects/{projectId}`

Logical source: **PROJECT** · Subcollections: roles, members, tasks, interests

Publicly readable when status != Draft (guests browse). A Draft is visible to its owner only.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Auto-ID
ownerId	string	req	Project owner · FK → users
title	string	req	
description	string	req	
purpose	string (enum project_purpose)	req	Learning / Real_World (mandatory)
projectType	string (enum project_type)	req	
track	string (enum track)	req	
difficulty	string (enum difficulty)	req	
status	string (enum project_status)	req	Draft / Open / Full / Completed / Archived
teamSizeTarget	number (int)	req	Target team size
expectedHoursPerWeek	number (int)	opt	Expected commitment (Real-world)
minLevel	string (enum level)	opt	Minimum level (Real-world)
mentoringAvailable	boolean	opt	Mentoring available (Learning)
expectedEndDate	timestamp (date @ 00:00 UTC)	opt	Expected end date
creationMode	string (enum creation_mode)	req	Manual / AI_Assisted
createdAt	timestamp	req	
publishedAt	timestamp	opt	Published only with the owner's approval
updatedAt	timestamp	req	
setupGenerationId **FK**	string	opt	FK -> aiGenerations (if AI setup was used)
ownerName **D**	string	req	Snapshot of USER.display_name
ownerPhotoUrl **D**	string	opt	Snapshot
techStack **D**	array<map>	req	PROJECT_SKILL: {skillId, name, isRequired}
techStackIds **D**	array<string>	req	Flat list for the Tech stack filter (array-contains-any)
requiredSkillIds **D**	array<string>	req	Required ones only (isRequired = true)
memberCount **D**	number	req	Number of active PROJECT_MEMBER rows
openRoleCount **D**	number	req	Roles that still have open slots (Missing Role Alert)
openRoleNames **D**	array<string>	req	Roles needed filter
interestCount **D**	number	req	Number of INTEREST rows (Trending)

B.3.12 projects/{projectId}/roles/{roleId}

Logical source: **PROJECT_ROLE**

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Auto-ID
roleName	string	req	
description	string	opt	
slotsNeeded	number (int)	req	Number of open places (default 1)
isAiGenerated	boolean	req	
createdAt	timestamp	req	
requiredSkillIds D	array<string>	opt	ROLE_SKILL
slotsFilled D	number	req	Members currently in the role

B.3.13 projects/{projectId}/members/{uid}

Logical source: **PROJECT_MEMBER**

The My Teams list = collection-group query on members filtered by userId.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Member uid (guarantees UNIQUE project + user)
memberType	string (enum member_type)	req	Owner / Member
status	string (enum member_status)	req	Active / Left / Removed
joinedAt	timestamp	req	
leftAt	timestamp	opt	
userId D	string	req	Copy of the document ID (needed for collection-group queries)
roleId FK	string	opt	FK -> roles
userName D	string	req	Snapshot
userPhotoUrl D	string	opt	Snapshot
roleName D	string	opt	Snapshot
projectTitle D	string	req	Snapshot (My Teams list)
projectPurpose D	string	req	Snapshot
projectStatus D	string	req	Snapshot

B.3.14 `projects/{projectId}/tasks/{taskId}`

Logical source: **TASK**

Offline writes are queued by the Firestore SDK (UC-05).

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Auto-ID
title	string	req	
description	string	opt	
status	string (enum task_status)	req	
priority	string (enum task_priority)	req	
deadline	timestamp (date @ 00:00 UTC)	opt	
assigneeId	string	opt	Assignee (must be a team member) · FK → users
creatorId	string	req	FK → users
createdAt	timestamp	req	
updatedAt	timestamp	req	
assigneeName **D**	string	opt	Snapshot

B.3.15 `projects/{projectId}/interests/{uid}`

Logical source: **INTEREST**

Field	Firestore type	Req	Source / notes
(document ID)	string	req	uid (guarantees UNIQUE)
createdAt	timestamp	req	
userId **D**	string	req	Copy of the document ID (collection-group)

B.3.16 `joinRequests/{projectId}_{applicantId}`

Logical source: **JOIN_REQUEST**

Top-level so that both the applicant and the owner can read it easily.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	{projectId}_{applicantId} - deterministic ID prevents duplicates
projectId	string	req	FK → projects
applicantId	string	req	Applicant · FK → users
roleId	string	req	Requested role · FK → project_roles
pitchMessage	string	req	Join message (AI-generated, editable)
status	string (enum request_status)	req	
createdAt	timestamp	req	
decidedAt	timestamp	opt	
pitchGenerationId FK	string	opt	FK -> aiGenerations
ownerId D	string	req	PROJECT.owner_id (for rules and Incoming Requests)
projectTitle D	string	req	Snapshot
applicantName D	string	req	Snapshot
applicantPhotoUrl D	string	opt	Snapshot
roleName D	string	req	Snapshot
matchScore D	number	opt	Snapshot at send time
matchedSkills D	array<string>	opt	Snapshot (Applicant Details)

B.3.17 `invitations/{projectId}_{inviteeId}`

Logical source: **INVITATION**

A new invitation after Declined / Cancelled overwrites the same document (assumption).

Field	Firestore type	Req	Source / notes
(document ID)	string	req	{projectId}_{inviteeId}
projectId	string	req	FK → projects
roleId	string	opt	FK → project_roles
inviteeId	string	req	Invited developer · FK → users
inviterId	string	req	Owner who sent the invitation · FK → users
message	string	opt	
status	string (enum invitation_status)	req	
createdAt	timestamp	req	
respondedAt	timestamp	opt	
projectTitle D	string	req	Snapshot
inviterName D	string	req	Snapshot
roleName D	string	opt	Snapshot

B.3.18 `conversations/{conversationId}`

Logical source: **CONVERSATION** · Subcollections: messages

The deterministic ID prevents duplicate direct conversations.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Direct: {uidA}_{uidB} (sorted) / Team: team_{projectId}
conversationType	string (enum conversation_type)	req	
projectId	string	opt	Team conversations only · FK → projects
createdAt	timestamp	req	
participantIds **D**	array<string>	req	CONVERSATION_PARTICIPANT.user_id (Inbox query with array-contains)
lastReadAt **D**	map<uid, timestamp>	opt	CONVERSATION_PARTICIPANT.last_read_at
lastMessage **D**	map	opt	{text, senderId, sentAt} - latest message
lastMessageAt **D**	timestamp	opt	Inbox ordering

B.3.19 `conversations/{conversationId}/messages/{messageId}`

Logical source: **MESSAGE**

Real-time listener.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Auto-ID
senderId	string	req	FK → users
body	string	req	
sentAt	timestamp	req	
editedAt	timestamp	opt	

B.3.20 `promptVersions/{promptVersionId}`

Logical source: **PROMPT_VERSION**

Server / admin only. The active version is also read from Remote Config.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Slug such as match_explain_v2
featureType	string (enum feature_type)	req	
versionLabel	string	req	v1, v2 ...
engine	string (enum engine)	req	LLM / RULE_BASED
template	string	opt	Prompt text
isActive	boolean	req	Currently active version (Remote Config)
createdAt	timestamp	req	
notes	string	opt	

B.3.21 promptEvalRuns/{runId}

Logical source: **PROMPT_EVAL_RUN**

Server / admin only - delivery evidence.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Auto-ID
promptVersionId	string	req	FK → prompt_versions
testSetLabel	string	req	Test set name
sampleCount	number (int)	req	
avgRelevance	number (double)	req	0 - 5
avgClarity	number (double)	req	Explanation clarity (target >= 4)
suitableRatePct	number (double)	req	% of recommendations rated suitable (target >= 60)
avgLatencyMs	number (int)	opt	
runAt	timestamp	req	
notes	string	opt	

B.3.22 aiGenerations/{generationId}

Logical source: **AI_GENERATION**

Server only (Admin SDK).

Field	Firestore type	Req	Source / notes
(document ID)	string	req	Auto-ID
userId	string	req	User who requested the generation · FK → users
promptVersionId	string	req	FK → prompt_versions
source	string (enum gen_source)	req	Live / Cached / Fallback (NFR-11)
latencyMs	number (int)	opt	Response time (NFR-01)
createdAt	timestamp	req	
featureType **D**	string	req	PROMPT_VERSION.feature_type (for aggregation)

B.3.23 aiFeedback/{generationId}_{userId}

Logical source: **AI_FEEDBACK**

A user writes only their own document.

Field	Firestore type	Req	Source / notes
(document ID)	string	req	{generationId}_{userId} - one rating per user
generationId	string	req	FK → ai_generations
userId	string	req	FK → users
isHelpful	boolean	req	Thumbs up / down
clarityRating	number (int)	opt	1 - 5
reason	string	opt	Reason for rejection
createdAt	timestamp	req	
promptVersionId **D**	string	req	From AI_GENERATION (to compute per-version metrics)
featureType **D**	string	req	From PROMPT_VERSION

B.4 Sample documents

users/{uid}

```
{
  "displayName": "Sample Developer",
  "publicId": "dev-7f3a91",
  "bio": "Flutter student",
  "mainTrack": "Mobile",
  "level": "Junior",
  "languages": [
    {
      "skillId": "dart",
      "name": "Dart",
      "level": "Intermediate"
    },
    {
      "skillId": "python",
      "name": "Python",
      "level": "Beginner"
    }
  ],
  "skills": [
    {
      "skillId": "flutter",
      "name": "Flutter"
    }
  ],
  "softSkills": [
    {
      "skillId": "teamwork",
      "name": "Teamwork"
    }
  ],
  "interests": [
    {
      "skillId": "ai",
      "name": "AI"
    }
  ],
  "skillIds": [
    "dart",
    "python",
    "flutter",
    "teamwork",
    "ai"
  ],
  "portfolioUrl": "https://example.com",
  "linkedinUrl": "https://linkedin.com/in/example",
  "githubConnected": true,
  "hasCv": true,
  "profileCompletion": 85,
  "authProvider": "google",
  "createdAt": "<timestamp>",
  "updatedAt": "<timestamp>"
}
```

projects/{projectId}

```
{
  "ownerId": "uidOwner",
  "ownerName": "Owner Name",
  "title": "Study Buddy",
  "description": "…",
  "purpose": "Learning",
  "projectType": "Mobile_App",
  "track": "Mobile",
  "difficulty": "Beginner",
  "status": "Open",
  "teamSizeTarget": 4,
  "expectedHoursPerWeek": null,
  "minLevel": null,
  "mentoringAvailable": true,
  "expectedEndDate": null,
  "creationMode": "AI_Assisted",
  "setupGenerationId": "gen_91",
  "techStack": [
    {
      "skillId": "flutter",
      "name": "Flutter",
      "isRequired": true
    }
  ],
  "techStackIds": [
    "flutter",
    "firebase"
  ],
  "requiredSkillIds": [
    "flutter"
  ],
  "memberCount": 1,
  "openRoleCount": 3,
  "openRoleNames": [
    "Frontend",
    "Backend",
    "QA"
  ],
  "interestCount": 5,
  "createdAt": "<timestamp>",
  "publishedAt": "<timestamp>",
  "updatedAt": "<timestamp>"
}
```

joinRequests/{projectId}_{applicantId}

```
{
  "projectId": "p_123",
  "applicantId": "uidApplicant",
  "ownerId": "uidOwner",
  "roleId": "r_backend",
  "roleName": "Backend",
  "pitchMessage": "Hi, I would like to join …",
  "pitchGenerationId": "gen_104",
  "status": "Pending",
  "projectTitle": "Study Buddy",
  "applicantName": "Sample Developer",
  "matchScore": 78,
  "matchedSkills": [
    "dart",
    "firebase"
  ],
  "createdAt": "<timestamp>",
  "decidedAt": null
}
```

users/{uid}/matches/{matchId}

```
{
  "projectId": "p_123",
  "projectTitle": "Study Buddy",
  "projectPurpose": "Learning",
  "generationId": "gen_88",
  "matchScore": 78,
  "explanationText": "You already know Dart and Firebase, which this project needs …",
  "source": "Live",
  "promptVersionId": "match_explain_v2",
  "matchedSkills": [
    {
      "skillId": "dart",
      "name": "Dart"
    }
  ],
  "complementarySkills": [],
  "crossSkills": [],
  "missingSkills": [
    {
      "skillId": "restapi",
      "name": "REST API",
      "learningTip": "Build a small CRUD client first."
    }
  ],
  "createdAt": "<timestamp>"
}
```

B.5 Composite indexes

Single-field indexes are automatic. These are the composite indexes that must be declared in `firestore.indexes.json` for the filters and sort orders. **Must** = needed for the MVP; **Should / Could** = needed if the feature is implemented.

#	Collection	Scope	Fields (order)	Serves	Priority
1	`projects`	Collection	`status asc, createdAt desc`	Discover - Newest	Must
2	`projects`	Collection	`status asc, interestCount desc`	Discover - Trending	Must
3	`projects`	Collection	`status asc, purpose asc, createdAt desc`	Project purpose filter (Learning / Real)	Must
4	`projects`	Collection	`status asc, difficulty asc, createdAt desc`	Difficulty filter	Must
5	`projects`	Collection	`status asc, track asc, createdAt desc`	Track filter	Must
6	`projects`	Collection	`status asc, projectType asc, createdAt desc`	Project type filter	Must
7	`projects`	Collection	`status asc, purpose asc, difficulty asc, createdAt desc`	Common combined filters	Should
8	`projects`	Collection	`status asc, teamSizeTarget asc, createdAt desc`	Team size filter	Should
9	`projects`	Collection	`techStackIds array-contains(-any), status asc, createdAt desc`	Tech stack filter	Must
10	`projects`	Collection	`openRoleNames array-contains, status asc, createdAt desc`	Roles needed filter	Should
11	`projects`	Collection	`ownerId asc, status asc, updatedAt desc`	My Projects (owner's Ongoing / Previous)	Must
12	`users`	Collection	`track asc, level asc, profileCompletion desc`	Discover Developers	Should
13	`users`	Collection	`skillIds array-contains, profileCompletion desc`	Developers with a given skill	Should
14	`joinRequests`	Collection	`ownerId asc, status asc, createdAt desc`	Incoming Requests	Must
15	`joinRequests`	Collection	`applicantId asc, status asc, createdAt desc`	My request status (Pending / Accepted / Rejected)	Must
16	`joinRequests`	Collection	`projectId asc, status asc, createdAt desc`	Requests of one project	Should
17	`invitations`	Collection	`inviteeId asc, status asc, createdAt desc`	Invited tab	Should
18	`invitations`	Collection	`projectId asc, status asc, createdAt desc`	Project invitations (owner)	Should
19	`members`	Collection group	`userId asc, status asc, joinedAt desc`	My Teams / Ongoing (member)	Must
20	`interests`	Collection group	`userId asc, createdAt desc`	Projects I expressed interest in	Could
21	`tasks`	Collection	`status asc, deadline asc`	Task Board by status	Must
22	`tasks`	Collection	`assigneeId asc, status asc, deadline asc`	Filter tasks by assignee / status	Must
23	`conversations`	Collection	`participantIds array-contains, lastMessageAt desc`	Messages Inbox	Should
24	`notifications`	Collection	`isRead asc, createdAt desc`	Unread badge + list	Must
25	`matches`	Collection	`projectId asc, createdAt desc`	Latest recommendation for a project (cache)	Should
26	`detectedSkills`	Collection	`status asc, confidence desc`	Review Detected Skills	Must

27	aiGenerations	Collection	promptVersionId asc, createdAt desc	Aggregate prompt versions	Must
28	aiFeedback	Collection	promptVersionId asc, createdAt desc	Prompt Evaluation metrics	Must

B.6 Denormalization register

Every denormalization in the physical schema has a reason (faster queries / simpler rules) and a refresh mechanism. The single source of truth remains the logical schema.

#	Physical field	Logical source	Reason	Maintained by	Staleness
1	`users.languages / skills / softSkills / interests`	USER_SKILL + SKILL	The developer profile is read in one read and sent to matching without joins.	User writes + Function when a Detected Skill is accepted	Skill name could change in the catalogue (rare) - a Function refreshes it
2	`users.skillIds`	USER_SKILL	array-contains for Discover Developers.	Same write	-
3	`users.githubConnected`	GITHUB_DATA	Avoids an extra read to show the connection state.	Function on Connect / Disconnect	-
4	`users.hasCv / private cvUrl`	USER.cv_url	The URL is not public (privacy).	Client on upload	-
5	`users.profileCompletion`	USER (derived)	Developer ordering + completion banner.	Client / Function on any edit	Must use the same formula everywhere
6	`githubData.repos[] / languages`	GITHUB_REPO	GitHub Insights screen needs a single read.	Function (Cloud Function that calls the GitHub API)	<= 30 repos because of the 1 MiB document limit
7	`detectedSkills.evidence[]`	SKILL_EVIDENCE	Evidence is always shown together with the skill.	Function	-
8	`projects.techStack / techStackIds / requiredSkillIds`	PROJECT_SKILL	Discover filters + card display without a subcollection.	Client on Create / Edit + Function	Skill name refreshed by a Function if it changes
9	`projects.ownerName / ownerPhotoUrl`	USER	Cards show the owner without an extra read.	Function onUpdate(users)	Stale until the Function finishes
10	`projects.memberCount / openRoleCount / openRoleNames`	PROJECT_MEMBER, PROJECT_ROLE	Status / Roles filters + Missing Role Alert.	Function triggers	Eventual consistency (seconds)
11	`projects.interestCount`	INTEREST	Trending sort.	Function onCreate / onDelete(interests)	Approximate
12	`roles.requiredSkillIds / slotsFilled`	ROLE_SKILL, PROJECT_MEMBER	Missing role + Skill gap without joins.	Function	-
13	`members.* snapshots (userName, projectTitle ...)`	USER, PROJECT	My Teams list in one query on a collection group.	Function onUpdate	Possibly stale
14	`tasks.assigneeName`	USER	Task Board without joins.	Client on assignment	-
15	`joinRequests.* snapshots (ownerId, applicantName, matchScore ...)`	PROJECT, USER, MATCH	Incoming Requests and Applicant Details in one read + rules.	Client + Function	The snapshot intentionally stays as it was at send time
16	`savedProjects.* snapshots`	PROJECT	Saved list without joins.	Function onUpdate(projects)	Possibly stale
17	`matches.*Skills / projectTitle / source`	MATCH_SKILL, PROJECT, AI_GENERATION	Match History + Why this fits in one read.	Cloud Function (at generation time)	Immutable after write
18	`conversations.participantIds / lastReadAt / lastMessage`	CONVERSATION_PARTICIPANT, MESSAGE	One ordered Inbox query by latest message.	Function onCreate(messages)	-

19	`aiFeedback.promptVersionId / featureType`	AI_GENERATION, PROMPT_VERSION	Aggregate per-version metrics from a single collection.	Client / Function at write time	Immutable

B.7 Security rules

Permission summary per collection (implements NFR-03: each user reaches only their own data), followed by a reference `firestore.rules` sample - it must be tested with the Emulator before it is relied on.

Path	Guest	Owner / self	Other users	Writes by	Notes
`skills`	read	read	-	Admin / Function	Public catalogue.
`users/{uid}`	read (public profile)	create / update: owner	read	Owner (profile fields) / Functions (profileCompletion, githubConnected)	No email / CV URL in this doc.
`users/{uid}/private/*`	no	owner read / write	no	Owner	email + cvUrl.
`users/{uid}/devices/*`	no	owner read / write	no	Owner	
`users/{uid}/githubData, detectedSkills`	no	owner read; update status only	no	Functions (all other fields)	Disconnect = delete via a Function.
`users/{uid}/savedProjects`	no	owner read / write	no	Owner	
`users/{uid}/matches, aiUsage`	no	owner read	no	Functions only	Protects the cost cap (R5).
`users/{uid}/notifications`	no	owner read + isRead only	no	Functions (create)	
`projects/{id}`	read if status != Draft	owner: full; reads own Draft	read (!= Draft)	Owner (create / update) / Functions (counters)	create requires ownerId == auth.uid.
`projects/{id}/roles`	read	owner write	read	Owner / Function	
`projects/{id}/members`	no	members read	members read	Function (accepting a request)	Joining happens only after the owner accepts.
`projects/{id}/tasks`	no	members read / create / update; owner delete	members only	Members	
`projects/{id}/interests`	no	owner of the doc: create / delete	project owner reads	Signed-in user (own doc)	
`joinRequests/{pid}_{uid}`	no	applicant: create (Pending) + read; owner: read + update status	no	Applicant (create) / Owner (decision)	Deterministic ID = once only.
`invitations/{pid}_{uid}`	no	project owner create / cancel; invitee read + respond	no	Owner / Invitee	
`conversations/*`	no	participants read / write	no	Participants (messages) / Function (metadata)	uid in participantIds.
`promptVersions, promptEvalRuns, aiGenerations`	no	no	no	Admin SDK only	No client access.
`aiFeedback/{gid}_{uid}`	no	create / read own doc	no	Owner	Deterministic ID = one rating.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    function signedIn() { return request.auth != null; }
    function isOwner(uid) { return signedIn() && request.auth.uid == uid; }
    function isMember(pid) {
      return signedIn() &&
        exists(/databases/$(db)/documents/projects/$(pid)/members/$(request.auth.uid));
    }

    match /skills/{id}  { allow read: if true; allow write: if false; }
    match /users/{uid} {
      allow read: if true;                       // public profile (guests included)
      allow create, update: if isOwner(uid);
      match /{sub=**} { allow read, write: if isOwner(uid); }   // tighten per subcollection
    }
    match /projects/{pid} {
      allow read: if resource.data.status != 'Draft' ||
                     isOwner(resource.data.ownerId);
      allow create: if signedIn() && request.resource.data.ownerId == request.auth.uid;
      allow update, delete: if isOwner(resource.data.ownerId);
      match /roles/{rid}   { allow read: if true;
                             allow write: if
isOwner(get(/databases/$(db)/documents/projects/$(pid)).data.ownerId); }
      match /members/{uid} { allow read: if isMember(pid); allow write: if false; }  // Functions only
      match /tasks/{tid}   { allow read, create, update: if isMember(pid);
                             allow delete: if
isOwner(get(/databases/$(db)/documents/projects/$(pid)).data.ownerId); }
    }
    match /joinRequests/{rid} {
      allow create: if signedIn()
        && rid == request.resource.data.projectId + '_' + request.auth.uid   // once per project
        && request.resource.data.applicantId == request.auth.uid
        && request.resource.data.status == 'Pending';
      allow read: if signedIn() && (resource.data.applicantId == request.auth.uid
                                 || resource.data.ownerId == request.auth.uid);
      allow update: if isOwner(resource.data.ownerId)
        && request.resource.data.diff(resource.data).affectedKeys().hasOnly(['status','decidedAt']);
    }
    match /conversations/{cid} {
      allow read: if signedIn() && request.auth.uid in resource.data.participantIds;
      match /messages/{mid} {
        allow read: if signedIn() &&
          request.auth.uid in get(/databases/$(db)/documents/conversations/$(cid)).data.participantIds;
        allow create: if request.resource.data.senderId == request.auth.uid;
      }
    }
    match /{col}/{id} { allow read, write: if false; }   // promptVersions, aiGenerations ... (Admin SDK only)
  }
}
```

B.8 Firebase Storage

Path	Content	Read	Write / notes
`avatars/{uid}/avatar.jpg`	Profile photo	public read	owner write (<= 2 MB, image/*)
`cv/{uid}/cv.pdf`	CV (PDF)	owner read / write only (<= 5 MB, application/pdf)	Owner. A project owner views an applicant's CV through a Cloud Function that checks a joinRequest exists and returns a short-lived signed URL.

Firestore stores only the CV path (in `users/{uid}/private/account.cvUrl`), not the file itself. A download URL containing a token is never stored in a public document because it bypasses Storage Rules.

B.9 Logical → Physical mapping

Logical table	Technique	Firestore location
users	Own collection	users/{uid}
user_devices	Own collection	users/{uid}/devices/{deviceId}
users (email, cv_url)	Vertical split	users/{uid}/private/account
skills	Own collection	skills/{skillId}
user_skills	Embedded	users.languages[] / skills[] / softSkills[] / interests[] + skillIds[]
github_data	Own collection	users/{uid}/githubData/summary
github_repos	Embedded	users/{uid}/githubData/summary.repos[]
detected_skills	Own collection	users/{uid}/detectedSkills/{skillId}
skill_evidence	Embedded	users/{uid}/detectedSkills/{skillId}.evidence[]
projects	Own collection	projects/{projectId}
project_skills	Embedded	projects.techStack[] + techStackIds[] + requiredSkillIds[]
project_roles	Own collection	projects/{projectId}/roles/{roleId}
role_skills	Embedded	projects/{projectId}/roles/{roleId}.requiredSkillIds[]
project_members	Own collection	projects/{projectId}/members/{uid}
saved_projects	Own collection	users/{uid}/savedProjects/{projectId}
interests	Own collection	projects/{projectId}/interests/{uid}
join_requests	Own collection	joinRequests/{projectId}_{applicantId}
invitations	Own collection	invitations/{projectId}_{inviteeId}
prompt_versions	Own collection	promptVersions/{promptVersionId}
ai_generations	Own collection	aiGenerations/{generationId}
match_recommendations	Own collection	users/{uid}/matches/{matchId}
match_skills	Embedded	users/{uid}/matches/{matchId}.matchedSkills[] / complementarySkills[] / crossSkills[] / missingSkills[]
ai_feedback	Own collection	aiFeedback/{generationId}_{userId}
prompt_eval_runs	Own collection	promptEvalRuns/{runId}
tasks	Own collection	projects/{projectId}/tasks/{taskId}
conversations	Own collection	conversations/{conversationId}
conversation_participants	Embedded	conversations/{conversationId}.participantIds[] + lastReadAt{}
messages	Own collection	conversations/{conversationId}/messages/{messageId}
notifications	Own collection	users/{uid}/notifications/{notificationId}
ai_usage_counters	Own collection	users/{uid}/aiUsage/{yyyyMMdd}

Part C
PostgreSQL reference DDL

A SQL (PostgreSQL) version of exactly the same logical schema - for reference, or if the supervisor asks for a traditional relational schema. It is generated from the same model (same columns and constraints).

C.1 DDL

```
-- DevSync — PostgreSQL reference DDL (generated from the logical schema)
-- IDs are VARCHAR so Firestore document IDs / Firebase UIDs can be migrated as-is.

CREATE TYPE track AS ENUM ('Mobile', 'Web', 'Backend', 'Data', 'AI', 'UI_UX', 'QA', 'DevOps', 'Other');
CREATE TYPE level AS ENUM ('Beginner', 'Junior', 'Intermediate', 'Advanced');
CREATE TYPE auth_provider AS ENUM ('password', 'google');
CREATE TYPE platform AS ENUM ('android');
CREATE TYPE skill_kind AS ENUM ('PROGRAMMING_LANGUAGE', 'TECHNICAL', 'SOFT', 'INTEREST');
CREATE TYPE skill_source AS ENUM ('manual', 'github');
CREATE TYPE evidence_type AS ENUM ('Repo', 'Commits', 'Pull_Request', 'Language_Usage');
CREATE TYPE detect_status AS ENUM ('pending', 'accepted', 'rejected');
CREATE TYPE project_purpose AS ENUM ('Learning', 'Real_World');
CREATE TYPE project_type AS ENUM ('Web_App', 'Mobile_App', 'API_Backend', 'Library_Tool', 'Data_AI', 'Game', 'Other');
CREATE TYPE difficulty AS ENUM ('Beginner', 'Intermediate', 'Advanced');
CREATE TYPE project_status AS ENUM ('Draft', 'Open', 'Full', 'Completed', 'Archived');
CREATE TYPE creation_mode AS ENUM ('Manual', 'AI_Assisted');
CREATE TYPE member_type AS ENUM ('Owner', 'Member');
CREATE TYPE member_status AS ENUM ('Active', 'Left', 'Removed');
CREATE TYPE request_status AS ENUM ('Pending', 'Accepted', 'Rejected');
CREATE TYPE invitation_status AS ENUM ('Pending', 'Accepted', 'Declined', 'Cancelled');
CREATE TYPE engine AS ENUM ('LLM', 'RULE_BASED');
CREATE TYPE feature_type AS ENUM ('SKILL_INFERENCE', 'MATCH_EXPLAIN', 'SKILL_GAP', 'PROJECT_SETUP', 'ROLE_GENERATOR',
'COLLAB_PITCH');
CREATE TYPE gen_source AS ENUM ('Live', 'Cached', 'Fallback');
CREATE TYPE match_type AS ENUM ('Matched', 'Complementary', 'Cross_Skill', 'Missing');
CREATE TYPE task_status AS ENUM ('To_Do', 'In_Progress', 'In_Review', 'Done');
CREATE TYPE task_priority AS ENUM ('Low', 'Medium', 'High');
CREATE TYPE conversation_type AS ENUM ('Direct', 'Team');
CREATE TYPE notification_type AS ENUM ('Join_Request_Received', 'Request_Accepted', 'Request_Rejected',
'Invitation_Received', 'Task_Assigned', 'Task_Updated', 'New_Message', 'Missing_Role');

CREATE TABLE users (
  uid                       VARCHAR(128) NOT NULL,
  email                     VARCHAR(255) NOT NULL UNIQUE,
  email_verified            BOOLEAN NOT NULL,
  auth_provider             auth_provider NOT NULL,
  display_name              VARCHAR(80) NOT NULL,
  public_id                 VARCHAR(20) NOT NULL UNIQUE,
  photo_url                 VARCHAR(500),
  bio                       TEXT,
  main_track                track,
  level                     level,
  background                TEXT,
  cv_url                    VARCHAR(500),
  portfolio_url             VARCHAR(500),
  linkedin_url              VARCHAR(500),
  behance_url               VARCHAR(500),
```

```
  created_at                TIMESTAMPTZ NOT NULL,
  updated_at                TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (uid)
);

CREATE TABLE user_devices (
  device_id                 VARCHAR(64) NOT NULL,
  user_id                   VARCHAR(128) NOT NULL,
  fcm_token                 VARCHAR(255) NOT NULL UNIQUE,
  platform                  platform NOT NULL,
  last_seen_at              TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (device_id),
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE
);

CREATE TABLE skills (
  skill_id                  VARCHAR(64) NOT NULL,
  name                      VARCHAR(80) NOT NULL,
  kind                      skill_kind NOT NULL,
  is_active                 BOOLEAN NOT NULL,
  PRIMARY KEY (skill_id),
  UNIQUE (name, kind)
);

CREATE TABLE user_skills (
  user_id                   VARCHAR(128) NOT NULL,
  skill_id                  VARCHAR(64) NOT NULL,
  proficiency               level,
  source                    skill_source NOT NULL,
  added_at                  TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (user_id, skill_id),
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE,
  FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE CASCADE
);

CREATE TABLE github_data (
  user_id                   VARCHAR(128) NOT NULL,
  github_username           VARCHAR(80) NOT NULL UNIQUE,
  repos_count               INTEGER NOT NULL,
  commits_count             INTEGER NOT NULL,
  pull_requests_count       INTEGER NOT NULL,
  last_activity_at          TIMESTAMPTZ,
  last_sync_at              TIMESTAMPTZ NOT NULL,
  sync_status               VARCHAR(20) NOT NULL,
  PRIMARY KEY (user_id),
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE
);

CREATE TABLE github_repos (
  repo_id                   VARCHAR(64) NOT NULL,
  user_id                   VARCHAR(128) NOT NULL,
  repo_name                 VARCHAR(120) NOT NULL,
  primary_language          VARCHAR(40),
  stars                     INTEGER NOT NULL,
  commit_count              INTEGER NOT NULL,
  last_pushed_at            TIMESTAMPTZ,
  repo_url                  VARCHAR(500) NOT NULL,
  PRIMARY KEY (repo_id),
  FOREIGN KEY (user_id) REFERENCES github_data(user_id) ON DELETE CASCADE,
  UNIQUE (user_id, repo_name)
);

CREATE TABLE detected_skills (
  detected_id               VARCHAR(64) NOT NULL,
  user_id                   VARCHAR(128) NOT NULL,
  skill_id                  VARCHAR(64) NOT NULL,
  confidence                NUMERIC(3,2) NOT NULL,
  last_active_at            TIMESTAMPTZ,
  status                    detect_status NOT NULL,
  detected_at               TIMESTAMPTZ NOT NULL,
  reviewed_at               TIMESTAMPTZ,
  PRIMARY KEY (detected_id),
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE,
  FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE CASCADE,
  UNIQUE (user_id, skill_id),
  CHECK (confidence BETWEEN 0 AND 1)
);

CREATE TABLE skill_evidence (
  evidence_id               VARCHAR(64) NOT NULL,
```

```
  detected_id               VARCHAR(64) NOT NULL,
  repo_id                   VARCHAR(64),
  evidence_type             evidence_type NOT NULL,
  summary                   VARCHAR(255) NOT NULL,
  commit_count              INTEGER,
  last_activity_at          TIMESTAMPTZ,
  PRIMARY KEY (evidence_id),
  FOREIGN KEY (detected_id) REFERENCES detected_skills(detected_id) ON DELETE CASCADE,
  FOREIGN KEY (repo_id) REFERENCES github_repos(repo_id) ON DELETE SET NULL
);

CREATE TABLE prompt_versions (
  prompt_version_id         VARCHAR(64) NOT NULL,
  feature_type              feature_type NOT NULL,
  version_label             VARCHAR(20) NOT NULL,
  engine                    engine NOT NULL,
  template                  TEXT,
  is_active                 BOOLEAN NOT NULL,
  created_at                TIMESTAMPTZ NOT NULL,
  notes                     TEXT,
  PRIMARY KEY (prompt_version_id),
  UNIQUE (feature_type, version_label)
);

CREATE TABLE ai_generations (
  generation_id             VARCHAR(64) NOT NULL,
  user_id                   VARCHAR(128) NOT NULL,
  prompt_version_id         VARCHAR(64) NOT NULL,
  source                    gen_source NOT NULL,
  latency_ms                INTEGER,
  created_at                TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (generation_id),
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE,
  FOREIGN KEY (prompt_version_id) REFERENCES prompt_versions(prompt_version_id) ON DELETE RESTRICT
);

CREATE TABLE ai_feedback (
  feedback_id               VARCHAR(64) NOT NULL,
  generation_id             VARCHAR(64) NOT NULL,
  user_id                   VARCHAR(128) NOT NULL,
  is_helpful                BOOLEAN NOT NULL,
  clarity_rating            INTEGER,
  reason                    TEXT,
  created_at                TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (feedback_id),
  FOREIGN KEY (generation_id) REFERENCES ai_generations(generation_id) ON DELETE CASCADE,
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE,
  UNIQUE (generation_id, user_id),
  CHECK (clarity_rating IS NULL OR clarity_rating BETWEEN 1 AND 5)
);

CREATE TABLE prompt_eval_runs (
  run_id                    VARCHAR(64) NOT NULL,
  prompt_version_id         VARCHAR(64) NOT NULL,
  test_set_label            VARCHAR(60) NOT NULL,
  sample_count              INTEGER NOT NULL,
  avg_relevance             NUMERIC(3,2) NOT NULL,
  avg_clarity               NUMERIC(3,2) NOT NULL,
  suitable_rate_pct         NUMERIC(5,2) NOT NULL,
  avg_latency_ms            INTEGER,
  run_at                    TIMESTAMPTZ NOT NULL,
  notes                     TEXT,
  PRIMARY KEY (run_id),
  FOREIGN KEY (prompt_version_id) REFERENCES prompt_versions(prompt_version_id) ON DELETE RESTRICT
);

CREATE TABLE ai_usage_counters (
  user_id                   VARCHAR(128) NOT NULL,
  usage_date                DATE NOT NULL,
  request_count             INTEGER NOT NULL,
  PRIMARY KEY (user_id, usage_date),
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE
);

CREATE TABLE projects (
  project_id                VARCHAR(64) NOT NULL,
  owner_id                  VARCHAR(128) NOT NULL,
  title                     VARCHAR(120) NOT NULL,
  description               TEXT NOT NULL,
  purpose                   project_purpose NOT NULL,
```

```
  project_type              project_type NOT NULL,
  track                     track NOT NULL,
  difficulty                difficulty NOT NULL,
  status                    project_status NOT NULL,
  team_size_target          INTEGER NOT NULL,
  expected_hours_per_week   INTEGER,
  min_level                 level,
  mentoring_available       BOOLEAN,
  expected_end_date         DATE,
  creation_mode             creation_mode NOT NULL,
  setup_generation_id       VARCHAR(64) UNIQUE,
  created_at                TIMESTAMPTZ NOT NULL,
  published_at              TIMESTAMPTZ,
  updated_at                TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (project_id),
  FOREIGN KEY (owner_id) REFERENCES users(uid) ON DELETE CASCADE,
  FOREIGN KEY (setup_generation_id) REFERENCES ai_generations(generation_id) ON DELETE SET NULL,
  CHECK (team_size_target >= 1),
  CHECK (purpose <> 'Real_World' OR (expected_hours_per_week IS NOT NULL AND min_level IS NOT NULL))
);

CREATE TABLE project_skills (
  project_id                VARCHAR(64) NOT NULL,
  skill_id                  VARCHAR(64) NOT NULL,
  is_required               BOOLEAN NOT NULL,
  PRIMARY KEY (project_id, skill_id),
  FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE,
  FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE CASCADE
);

CREATE TABLE project_roles (
  role_id                   VARCHAR(64) NOT NULL,
  project_id                VARCHAR(64) NOT NULL,
  role_name                 VARCHAR(60) NOT NULL,
  description               TEXT,
  slots_needed              INTEGER NOT NULL,
  is_ai_generated           BOOLEAN NOT NULL,
  created_at                TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (role_id),
  FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE,
  UNIQUE (project_id, role_name),
  CHECK (slots_needed >= 1)
);

CREATE TABLE role_skills (
  role_id                   VARCHAR(64) NOT NULL,
  skill_id                  VARCHAR(64) NOT NULL,
  PRIMARY KEY (role_id, skill_id),
  FOREIGN KEY (role_id) REFERENCES project_roles(role_id) ON DELETE CASCADE,
  FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE CASCADE
);

CREATE TABLE project_members (
  project_id                VARCHAR(64) NOT NULL,
  user_id                   VARCHAR(128) NOT NULL,
  role_id                   VARCHAR(64),
  member_type               member_type NOT NULL,
  status                    member_status NOT NULL,
  joined_at                 TIMESTAMPTZ NOT NULL,
  left_at                   TIMESTAMPTZ,
  PRIMARY KEY (project_id, user_id),
  FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE,
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE,
  FOREIGN KEY (role_id) REFERENCES project_roles(role_id) ON DELETE SET NULL
);

CREATE TABLE saved_projects (
  user_id                   VARCHAR(128) NOT NULL,
  project_id                VARCHAR(64) NOT NULL,
  saved_at                  TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (user_id, project_id),
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE,
  FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE
);

CREATE TABLE interests (
  user_id                   VARCHAR(128) NOT NULL,
  project_id                VARCHAR(64) NOT NULL,
  created_at                TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (user_id, project_id),
```

```
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE,
  FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE
);

CREATE TABLE join_requests (
  request_id                VARCHAR(64) NOT NULL,
  project_id                VARCHAR(64) NOT NULL,
  applicant_id              VARCHAR(128) NOT NULL,
  role_id                   VARCHAR(64) NOT NULL,
  pitch_message             TEXT NOT NULL,
  pitch_generation_id       VARCHAR(64) UNIQUE,
  status                    request_status NOT NULL,
  created_at                TIMESTAMPTZ NOT NULL,
  decided_at                TIMESTAMPTZ,
  PRIMARY KEY (request_id),
  FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE,
  FOREIGN KEY (applicant_id) REFERENCES users(uid) ON DELETE CASCADE,
  FOREIGN KEY (role_id) REFERENCES project_roles(role_id) ON DELETE RESTRICT,
  FOREIGN KEY (pitch_generation_id) REFERENCES ai_generations(generation_id) ON DELETE SET NULL,
  UNIQUE (project_id, applicant_id)
);

CREATE TABLE invitations (
  invitation_id             VARCHAR(64) NOT NULL,
  project_id                VARCHAR(64) NOT NULL,
  role_id                   VARCHAR(64),
  invitee_id                VARCHAR(128) NOT NULL,
  inviter_id                VARCHAR(128) NOT NULL,
  message                   TEXT,
  status                    invitation_status NOT NULL,
  created_at                TIMESTAMPTZ NOT NULL,
  responded_at              TIMESTAMPTZ,
  PRIMARY KEY (invitation_id),
  FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE,
  FOREIGN KEY (role_id) REFERENCES project_roles(role_id) ON DELETE SET NULL,
  FOREIGN KEY (invitee_id) REFERENCES users(uid) ON DELETE CASCADE,
  FOREIGN KEY (inviter_id) REFERENCES users(uid) ON DELETE CASCADE
);

CREATE TABLE match_recommendations (
  match_id                  VARCHAR(64) NOT NULL,
  user_id                   VARCHAR(128) NOT NULL,
  project_id                VARCHAR(64) NOT NULL,
  generation_id             VARCHAR(64) NOT NULL UNIQUE,
  match_score               INTEGER NOT NULL,
  explanation_text          TEXT NOT NULL,
  created_at                TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (match_id),
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE,
  FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE,
  FOREIGN KEY (generation_id) REFERENCES ai_generations(generation_id) ON DELETE CASCADE,
  CHECK (match_score BETWEEN 0 AND 100)
);

CREATE TABLE match_skills (
  match_id                  VARCHAR(64) NOT NULL,
  skill_id                  VARCHAR(64) NOT NULL,
  match_type                match_type NOT NULL,
  learning_tip              TEXT,
  PRIMARY KEY (match_id, skill_id),
  FOREIGN KEY (match_id) REFERENCES match_recommendations(match_id) ON DELETE CASCADE,
  FOREIGN KEY (skill_id) REFERENCES skills(skill_id) ON DELETE CASCADE,
  CHECK (learning_tip IS NULL OR match_type = 'Missing')
);

CREATE TABLE tasks (
  task_id                   VARCHAR(64) NOT NULL,
  project_id                VARCHAR(64) NOT NULL,
  title                     VARCHAR(150) NOT NULL,
  description               TEXT,
  status                    task_status NOT NULL,
  priority                  task_priority NOT NULL,
  deadline                  DATE,
  assignee_id               VARCHAR(128),
  creator_id                VARCHAR(128) NOT NULL,
  created_at                TIMESTAMPTZ NOT NULL,
  updated_at                TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (task_id),
  FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE CASCADE,
  FOREIGN KEY (assignee_id) REFERENCES users(uid) ON DELETE SET NULL,
```

```
  FOREIGN KEY (creator_id) REFERENCES users(uid) ON DELETE CASCADE
);

CREATE TABLE conversations (
  conversation_id           VARCHAR(64) NOT NULL,
  conversation_type         conversation_type NOT NULL,
  project_id                VARCHAR(64) UNIQUE,
  created_at                TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (conversation_id),
  FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE SET NULL,
  CHECK ((conversation_type = 'Team') = (project_id IS NOT NULL))
);

CREATE TABLE conversation_participants (
  conversation_id           VARCHAR(64) NOT NULL,
  user_id                   VARCHAR(128) NOT NULL,
  joined_at                 TIMESTAMPTZ NOT NULL,
  last_read_at              TIMESTAMPTZ,
  PRIMARY KEY (conversation_id, user_id),
  FOREIGN KEY (conversation_id) REFERENCES conversations(conversation_id) ON DELETE CASCADE,
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE
);

CREATE TABLE messages (
  message_id                VARCHAR(64) NOT NULL,
  conversation_id           VARCHAR(64) NOT NULL,
  sender_id                 VARCHAR(128) NOT NULL,
  body                      TEXT NOT NULL,
  sent_at                   TIMESTAMPTZ NOT NULL,
  edited_at                 TIMESTAMPTZ,
  PRIMARY KEY (message_id),
  FOREIGN KEY (conversation_id) REFERENCES conversations(conversation_id) ON DELETE CASCADE,
  FOREIGN KEY (sender_id) REFERENCES users(uid) ON DELETE CASCADE
);

CREATE TABLE notifications (
  notification_id           VARCHAR(64) NOT NULL,
  user_id                   VARCHAR(128) NOT NULL,
  type                      notification_type NOT NULL,
  title                     VARCHAR(120) NOT NULL,
  body                      VARCHAR(300) NOT NULL,
  project_id                VARCHAR(64),
  deep_link                 VARCHAR(255),
  is_read                   BOOLEAN NOT NULL,
  created_at                TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (notification_id),
  FOREIGN KEY (user_id) REFERENCES users(uid) ON DELETE CASCADE,
  FOREIGN KEY (project_id) REFERENCES projects(project_id) ON DELETE SET NULL
);

-- Partial unique index: one pending invitation per (project, invitee)
CREATE UNIQUE INDEX uq_invitations_pending ON invitations (project_id, invitee_id) WHERE status = 'Pending';

-- Foreign-key indexes
CREATE INDEX idx_user_devices_user_id ON user_devices (user_id);
CREATE INDEX idx_user_skills_skill_id ON user_skills (skill_id);
CREATE INDEX idx_detected_skills_skill_id ON detected_skills (skill_id);
CREATE INDEX idx_skill_evidence_detected_id ON skill_evidence (detected_id);
CREATE INDEX idx_skill_evidence_repo_id ON skill_evidence (repo_id);
CREATE INDEX idx_ai_generations_user_id ON ai_generations (user_id);
CREATE INDEX idx_ai_generations_prompt_version_id ON ai_generations (prompt_version_id);
CREATE INDEX idx_ai_feedback_user_id ON ai_feedback (user_id);
CREATE INDEX idx_prompt_eval_runs_prompt_version_id ON prompt_eval_runs (prompt_version_id);
CREATE INDEX idx_projects_owner_id ON projects (owner_id);
CREATE INDEX idx_project_skills_skill_id ON project_skills (skill_id);
CREATE INDEX idx_role_skills_skill_id ON role_skills (skill_id);
CREATE INDEX idx_project_members_user_id ON project_members (user_id);
CREATE INDEX idx_project_members_role_id ON project_members (role_id);
CREATE INDEX idx_saved_projects_project_id ON saved_projects (project_id);
CREATE INDEX idx_interests_project_id ON interests (project_id);
CREATE INDEX idx_join_requests_applicant_id ON join_requests (applicant_id);
CREATE INDEX idx_join_requests_role_id ON join_requests (role_id);
CREATE INDEX idx_invitations_project_id ON invitations (project_id);
CREATE INDEX idx_invitations_role_id ON invitations (role_id);
CREATE INDEX idx_invitations_invitee_id ON invitations (invitee_id);
CREATE INDEX idx_invitations_inviter_id ON invitations (inviter_id);
CREATE INDEX idx_match_recommendations_user_id ON match_recommendations (user_id);
CREATE INDEX idx_match_recommendations_project_id ON match_recommendations (project_id);
CREATE INDEX idx_match_skills_skill_id ON match_skills (skill_id);
CREATE INDEX idx_tasks_project_id ON tasks (project_id);
```

```
CREATE INDEX idx_tasks_assignee_id ON tasks (assignee_id);
CREATE INDEX idx_tasks_creator_id ON tasks (creator_id);
CREATE INDEX idx_conversation_participants_user_id ON conversation_participants (user_id);
CREATE INDEX idx_messages_conversation_id ON messages (conversation_id);
CREATE INDEX idx_messages_sender_id ON messages (sender_id);
CREATE INDEX idx_notifications_user_id ON notifications (user_id);
CREATE INDEX idx_notifications_project_id ON notifications (project_id);

-- Query indexes (mirror the Firestore composite indexes)
CREATE INDEX idx_projects_discover   ON projects (status, purpose, difficulty, track, created_at DESC);
CREATE INDEX idx_projects_owner      ON projects (owner_id, status, updated_at DESC);
CREATE INDEX idx_requests_applicant  ON join_requests (applicant_id, status, created_at DESC);
CREATE INDEX idx_members_user        ON project_members (user_id, status, joined_at DESC);
CREATE INDEX idx_tasks_board         ON tasks (project_id, status, deadline);
CREATE INDEX idx_notifications_user  ON notifications (user_id, is_read, created_at DESC);
CREATE INDEX idx_messages_conv       ON messages (conversation_id, sent_at);
CREATE INDEX idx_gen_prompt          ON ai_generations (prompt_version_id, created_at DESC);
```

D Assumptions & Open Points

- Firestore is the actual database (per the architecture in the Comprehensive Analysis). The SQL is for reference only.
- A Guest is not an entity and nothing is stored for it.
- The GitHub token is not stored in Firestore (NFR-07) - secure storage or Secret Manager via a Cloud Function. The logical schema deliberately has no token column.
- The CV: the file lives in Firebase Storage and only its path is kept in `users/{uid}/private/account` (the document says Firestore stores the URL; it was moved to a private document because the users document is public to Guests).
- `users/{uid}/githubData` is treated as a subcollection holding a single document named `summary` (the document gives the path without a document ID).
- Enum values not found in the documents (project_type, difficulty, notification_type, feature_type, skill_kind) are proposals to be confirmed by the team.
- Trending = ordering by interestCount (the definition is not given in the documents).
- The Tech stack filter (array-contains-any) and the Roles needed filter (array-contains) cannot be combined in one Firestore query; the second filter is applied client-side or in a Cloud Function.
- Request to Join is allowed once per (project, applicant) for good (deterministic ID). To allow re-applying after a rejection, change the ID to an auto-ID and add a partial UNIQUE on Pending.
- Invitations use the deterministic ID `{projectId}_{inviteeId}`: a new invitation after Declined / Cancelled overwrites the old one. The logical schema allows an invitation history (partial unique on Pending).
- Delete account cascades to all user data (Cloud Function). What happens to projects that still have members (ownership transfer) is not defined in the documents.
- The daily cap on AI requests (ai_usage_counters) has no number defined (Risk R5 mentions a cap without a value).
- Free / Pro flavors are a build-time decision, so there is no subscriptions table. If real payments are defined, a SUBSCRIPTION table would be added.
- Cloud Functions require the Firebase Blaze plan (decision point 6) - needed for the denormalization triggers and for writing matches / counters.
- Features 7 (Community) and 13 (Chatbot) are excluded from this scope, so the schema has no posts, comments, likes, tags, reports or chatbot-message structures.